dida-trade: State-Machine Replication with Multi-Paxos, Fast Paxos and Vertical Paxos

Diogo Noronha da Cruz, n.º 106591 João Pedro Correia e Sousa, n.º 99991
Tomás Dias Monteiro, n.º 106211
Instituto Superior Técnico, Universidade de Lisboa
Design and Implementation of Distributed Applications 2026/27 — Group 38

Abstract

We replicate a small trading service with state-machine replication on top of a log of Paxos instances. The replicas run Multi-Paxos with pipelined phase 2, switch to Fast Paxos in the configurations that request it and fall back to classic ballots automatically on collisions, answer `balance` *outside consensus while preserving causality, and change their acceptor set at run time following Vertical Paxos, with state transfer to new and recovering replicas. This report explains the algorithms as implemented, the choices we made where the specification left room, and what each choice costs.*

1. Introduction

dida-trade orders `populate`, `add`, `buy` and `sell` commands through a replicated log [5]: position i of the log is chosen by the i-th instance of Paxos [2], and every replica executes the log strictly in order, so all replicas go through the same sequence of states. Clients send each request to every replica and accept the first reply. The work was done in five steps over a provided baseline; each of the following sections covers one of them, focusing on the decisions that the specification left open.

The implementation is in Java with gRPC. A replica has four parts: the client-facing service, which records a request as *pending* and blocks until it is executed; the Paxos service, which plays acceptor and learner; the *main loop*, which plays proposer/leader and executes decided positions; and the trade state machine. A request is identified by $\texttt{reqid} = 100 \cdot seq + client$, which makes duplicates harmless: if the same request ends up decided in two positions, the second one is skipped on execution.

2. Completing the baseline (Step 1)

Phase 1. The baseline leader considered itself elected without collecting a quorum of promises. Our `PhaseOneProcessor` declares phase 1 successful only when, for *every* ballot between the last complete ballot and the new one, a majority of that ballot's acceptors has promised. The schedules with a fixed acceptor set do not need more than this, but the same code is reused unchanged by Vertical Paxos in Step 5. A single refusal ends phase 1 at once: the refusing acceptor has promised a higher ballot, so there is no point waiting for the others, and the leader adopts that ballot. If the replies run out without a quorum and nobody refused (acceptors unreachable), the leader retries every second.

Debug modes. Freeze and slow mode are implemented as a gate at the start of every client and Paxos RPC handler. A frozen replica blocks the handler thread on a monitor instead of dropping the request, so all held requests resume, in their original threads, on un-freeze. The gate never holds the replica's lock while sleeping in slow mode, otherwise one delayed request would stall every other thread. The console service is never gated, because a frozen replica must still receive its un-freeze. Crash calls `Runtime.halt` after 200 ms, so the reply to the console leaves first and no shutdown hooks run, as in a real crash.

3. Multi-Paxos (Step 2)

One phase 1 per ballot. A new leader runs a single phase 1 starting at its first unexecuted position f. Each acceptor replies with every vote it cast (value and ballot) at positions $\geq f$, and with every value it knows to be decided, flagged as such. For each position the leader re-proposes the decided value if one was reported, otherwise the value voted in the highest ballot; gaps below the highest reported position are filled with no-ops. From then on, phase 2 is the only phase needed for any number of instances until the

ballot changes.

Pipelining. Each pending command gets its own proposer thread, which reserves the next free position under a short lock and sends phase 2 asynchronously. The leader never waits for a position to be decided before proposing the next one. Acceptors send their vote directly to all learners, so a decision costs one message delay after phase 2 instead of two (no round-trip through the leader), at the price of n^2 learn messages per instance. Learners count *distinct acceptors per value*, which costs nothing in classic ballots and is required in fast ones (Section 4).

Strict in-order execution. Positions are executed in order. A decided position whose command has not reached this replica yet blocks execution there instead of being skipped. That keeps all replicas identical, and in Step 5 it becomes the trigger for state transfer.

Preemption. Every refusal carries the acceptor's highest promised ballot. A leader that sees a higher ballot stops proposing. If it is the leader of the new ballot, its main loop runs phase 1 again.

4. Fast Paxos (Step 3)

In a fast ballot [3] the leader runs phase 1 as above. Once the old positions are recovered, it sends an *any* message opening every position from the first free one onwards. After that any replica may propose directly to the acceptors. Since clients already send every request to all replicas, every replica that received the *any* acts as a fast proposer.

Who chooses the position. In a fast round a proposer cannot reserve a position, because there is no leader to serialize reservations. We let the *acceptor* choose: a fast proposal carries no position, and each acceptor places the command in its lowest open position. A command already voted in this ballot keeps its position, so repeated deliveries are idempotent. A position holding only a vote from an older ballot counts as open, because the *any* only covers positions where phase 1 found that nothing could have been chosen. If all acceptors receive the same commands in the same order, they all pick the same positions and every command is decided after one round. If two replicas' proposals reach the acceptors in different orders, the same position receives different commands: a *collision*.

Quorums and decisions. With n acceptors we use classic quorums of $\lfloor n/2 \rfloor + 1$ and fast quorums of $\lceil 3n/4 \rceil$ (3 of 3, 4 of 5), which satisfy the Fast Paxos requirement that any classic quorum and any two fast quorums intersect. A learner decides a value when a fast quorum of distinct acceptors voted for that same value in the ballot.

Collision recovery and automatic reconfiguration. A learner detects a collision as soon as no value can still reach a fast quorum: the most voted value plus the acceptors that have not voted fall short of F. It then moves to ballot $b+1$, which is the automatic reconfiguration of the specification: every process is a learner, so every process can see that nothing can be chosen in b. Because all learners see the same votes, the leader of $b+1$ reaches the same conclusion without a message from anyone. A proposer also moves on if its fast proposals stay undecided for 3 s, which covers votes that will never arrive, for example from a crashed acceptor.

The leader of $b+1$ must then pick, for each position, a value that is safe even if b was fast. If k is the highest ballot voted in by the phase 1 quorum Q, a value v may have been chosen in a fast k only if some fast quorum R voted v. In that case all of $Q \cap R$ voted v, so at least $|Q| + F - n$ acceptors of Q voted v in k. If some value reaches that threshold the leader must re-propose it, otherwise nothing was chosen and any reported value is safe.

A race we had to handle. A fast proposal can reach an acceptor before the leader's *any* does. This becomes likely with slow mode, where the *any* may be delayed by up to a second. The acceptor must refuse it, since without the *any* it cannot know that the position is free. Without a retry, that acceptor's vote would never arrive and the fast quorum would be lost until the 3 s timeout. Proposals refused with no higher ballot are retried every 100 ms, and an *any* that fails to reach an acceptor is resent as well.

Trade-offs. In our deployment Fast Paxos does *not* save message delays. Every replica already receives each client request, so the classic leader and a fast proposer are equally far from the acceptors (client $\rightarrow$ replica $\rightarrow$ acceptors $\rightarrow$ learners, three hops in both cases). What Fast Paxos removes is the leader as a serialization point: a slow or frozen leader no longer delays commands once the *any* is out. The cost is collisions with concurrent clients. Each collision costs a ballot change and a new phase 1, which a classic leader never pays. With two concurrent clients on Schedule A, collisions appeared within the first few positions (Section 7). Fast quorums are also larger. With three acceptors $F = 3$, so a fast ballot cannot decide anything while one acceptor is down. Schedule A gets around this by alternating fast and classic ballots, whereas Schedule B (all ballots fast) makes no progress until the replica returns. A true two-hop fast path would have each acceptor vote locally on the request it received from the client, without forwarding it; we kept forwarding because it lets a replica that missed the client's message still vote.

5. Causally ordered `balance` (Step 4)

Every write goes through one totally ordered log, so a client's causal past can be summarized by a single number: how many log positions it has observed. This is a vector clock with a single entry. Each write reply carries the number of positions the replying replica had executed when the

write ran. The client keeps the maximum it has seen and sends it as `min_slot` with `balance`. A replica answers `balance` locally, without consensus, but only once it has executed at least `min_slot` positions. Until then it waits rather than return a stale value. The reply also carries the replica's position count, which the client folds into its clock.

This gives read-your-writes and monotonic reads: anything the client has observed is at a position below `min_slot`. It also gives writes-follow-reads: a write issued after a read can only be decided at a position above every position the read reflected, since those were already decided. State updates and the position counter change under the same lock that `balance` reads them with, so a reply can never pair the state after a command with the counter from before it, which would let the client later accept an older state.

Compared with ordering `balance` through Paxos, a read costs no consensus round and no log position, and the first replica to answer wins, so a single lagging replica is not on the critical path. The price is that two clients may see different, though causally consistent, values at the same moment, which the specification allows.

6. Dynamic membership and state transfer (Step 5)

Configurations follow Vertical Paxos [4]: each ballot has its own acceptor set and leader, given by the schedule, and in Schedule C the acceptors change from $\{0..4\}$ to $\{2..6\}$ at Config 2. The difficulty is that a leader must still find every value that may have been chosen, even when the acceptors that chose it are no longer in the configuration.

Phase 1 across configurations. The leader of ballot b runs phase 1 over the acceptors of every ballot from the last *complete* ballot c up to b, and needs a quorum of each set. This is the Vertical Paxos I rule, and the reason `PhaseOneProcessor` counts quorums per ballot.

When is a ballot complete? The baseline guessed c on the learner side from the ballots of decided positions. That is unsafe as soon as the acceptor set changes. A position decided in b does not imply that the *other* positions recovered by b's leader have reached b's acceptors. A later leader that only asks acceptors from b on could then miss a value chosen earlier by acceptors that have since left. We changed the rule: ballot b is complete only when (i) its leader finished phase 1, (ii) every recovered position, including those the leader already knew were decided, is accepted by a quorum of b's acceptors, and (iii) if the acceptor set changed, a quorum of b's acceptors has also received the decided log below the phase 1 starting point (`installstate`). After that, any quorum of b's acceptors knows everything that was ever decided, and earlier configurations can be dropped. Only the leader of b can establish this. It reports $c = b$ in its reply to the console, and the console (the Vertical Paxos master) passes it to the next leader with `ballot` or `activate`. A replica that never hears of a complete ballot keeps a lower c, which only makes its phase 1 ask more acceptors: it is never unsafe. For the same reason phase 1 now runs as soon as a replica becomes leader, without waiting for a client request, so that the console's `ballot` command completes promptly.

State transfer to learners. A replica cannot execute a decided position if it missed the learn messages, or if the client request never reached it, for example because it was down. When execution has been blocked for 1 s, the replica asks every peer (`fetchstate`) for the decided positions from its blocking point onwards, *with the command bodies*. These are installed as if they had been learned and executed normally. We transfer the log rather than a snapshot of the trade state. Replaying is deterministic and reuses the execution path. It also rebuilds the request-id set needed to deduplicate and to answer a client whose request arrives after the command was already installed. A snapshot would have to serialize the pending buy and sell order queues as well. Log transfer grows with history, which a production system would bound with checkpoints and log truncation [1].

Recovering replicas. A replica that restarts has lost its log, its state and, more dangerously, its acceptor promises and votes. Before promising or voting it fetches the log from its peers and replays it. If the peers show that the system was already running, it also *abstains* as an acceptor from every ballot up to the highest ballot they report. It may have voted in those ballots before crashing, and voting again could contradict an earlier vote, especially in fast ballots where acceptors pick positions. Abstaining is reported as "unavailable" rather than as a refusal. That way the remaining acceptors can still form classic quorums, and a fast ballot that needs the abstaining replica stalls and moves on to a ballot it can join. This is a heuristic substitute for stable storage. It relies on some peer having seen any ballot the crashed replica took part in, which holds in our setting because every ballot needs a quorum, but it is not a proof.

A gRPC detail. While a replica is down, its peers' channels to it enter gRPC's exponential reconnect backoff, which can grow to two minutes. During that time every call to the restarted replica fails immediately, and the replica could neither vote nor learn. A peer that receives `fetchstate` from a recovering replica therefore resets its back-off towards it.

A schedule bug. `ScheduleC` used classic quorums of 2 for the five acceptors of Configs 0–1. Two disjoint quorums such as $\{0, 1\}$ and $\{2, 3\}$ could then choose different values. We use 3 of 5 in every ballot.

7. Evaluation

We tested each schedule with all processes on one host, running scripted clients and console commands, and checked that every replica executed the same sequence of (position, command) pairs. A restarted replica is compared on the log it replays after restarting. The numbers below are from one run of each scenario.

Schedule A. With two concurrent clients (15 random orders each), the learners detected collisions in several positions of fast ballot 0. The system moved to classic ballot 1 without the console, and the three replicas executed identical 33-command logs. We then crashed replica 2, issued 10 more commands, and restarted it. It fetched the decided log from its peers, abstained from ballot 1, replayed the same 45-command log as the others, and a later causal `balance` returned a value that reflected the replayed writes.

Schedule B. The same scenario passed, except that no command could be decided while replica 2 was down ($F = 3$ of 3 in every ballot). The replicas kept moving ballots until replica 2 returned, and all pending commands then executed.

Schedule C. After commands in fast ballot 0, the console moved to ballot 2. Leader 3 pushed the 13 decided positions to $\{2..6\}$ and replied *completed* $= 2$. We then killed replicas 0 and 1, which have left the acceptor set, and the system kept deciding. The console's next `ballot 3 4` carried $c = 2$, so leader 4's phase 1 asked only $\{2..6\}$. Finally replica 6 was killed, missed 6 commands and caught up on restart. All live replicas executed identical 35-command logs.

Debugging these scenarios found two liveness bugs from earlier steps. The fast stall timer could never fire when votes arrived while the main loop was asleep. And acceptors that started a few milliseconds after the leader never received its *any*, so the first fast ballot of Schedule C never decided. Both are now fixed.

8. Conclusion

The resulting system orders writes with Multi-Paxos and pipelined phase 2, uses Fast Paxos where configured and falls back to classic ballots automatically on collisions or missing acceptors. Reads are causally consistent without consensus. Acceptor sets change at run time under Vertical Paxos, with an explicit and safe notion of ballot completion, log-based state transfer, and recovery of crashed replicas. Its main limitations are the lack of stable storage, which recovery works around by abstaining, and a transferred log that grows without bound.

Use of AI

"All design choices and the code have been by us. The following tools [list the tools] have been used for this purpose [list the purpose]."

Authorship

We, Diogo Noronha da Cruz, n.º 106591, MEIC; João Pedro Correia e Sousa, n.º 99991, MEIC; Tomás Dias Monteiro, n.º 106211, MEIC, hereby declare, on our honor, that the work entitled "dida-trade", carried out as part of the Design and Implementation of Distributed Applications course, resulted from the active participation of the following group members. The individual contributions were: Diogo Cruz — [summary of contribution]; João Sousa — [summary of contribution]; Tomás Monteiro — [summary of contribution]. We declare that this work is original and complies with the applicable academic regulations.

Lisboa, [date] Signatures:

References

[1] T. D. Chandra, R. Griesemer, and J. Redstone. Paxos made live: An engineering perspective. In *Proceedings of the 26th ACM Symposium on Principles of Distributed Computing (PODC)*, pages 398–407, 2007.

[2] L. Lamport. Paxos made simple. *ACM SIGACT News*, 32(4):51–58, 2001.

[3] L. Lamport. Fast Paxos. *Distributed Computing*, 19(2):79–103, 2006.

[4] L. Lamport, D. Malkhi, and L. Zhou. Vertical Paxos and primary-backup replication. In *Proceedings of the 28th ACM Symposium on Principles of Distributed Computing (PODC)*, pages 312–313, 2009.

[5] F. B. Schneider. Implementing fault-tolerant services using the state machine approach: A tutorial. *ACM Computing Surveys*, 22(4):299–319, 1990.